

ELEC 475 Lab 3

The plots/weights can be found in the Google Drive link in the weights_link.txt file.

Network Architectures

Encoder

For the encoder, we used MobileNetV3-Small, it was chosen because of its low parameter count and efficiency with limited resources. The feature taps are as shown:

```
self.low_level_idx = 1 # stride ~4, 16 channels  
  
self.mid_level_idx = 3 # stride ~8, 24 channels  
  
self.high_level_idx = 12 # stride ~16, 576 channels
```

Decoder

The feature map passes through an ASPP module with dilation rates {1, 6, 12, 18}. This is followed with projected low- and mid-level features, each reduced to 48 channels via 1×1 convs, followed by two 3×3 convolutions, BatchNorm, ReLU, and Dropout (0.5). A final 1×1 classifier outputs 21 semantic classes (VOC 2012), upsampled bilinearly to 256×256.

Knowledge Distillation

We trained three versions of the same student; baseline (no KD), response based KD, and feature based KD. The training flow is outlined as follows.

1. Load teacher = fcn_resnet50(pretrained=True)
2. Wrap with TeacherFeatureExtractor to capture layer1/layer2/layer3 outputs.
3. For each epoch:
 - a. Forward student + teacher.
 - b. Compute KD loss (response of feature) and CE loss.
 - c. Back-propagate student loss.
 - d. Validate and save best checkpoint.

Response-based distillation uses the Kullback-Leibler divergence between the softened teacher probabilities and the student outputs. Its loss is defined by this expression:

$$L_{response} = \alpha \cdot \mathcal{H}(\sigma(z_s; 1), y) + \beta \cdot \mathcal{H}(\sigma(z_s; \tau), \sigma(z_t; \tau))$$

Where:

- z_s = student logits
- z_t = teacher logits
- y = ground-truth labels
- $\sigma(z; \tau)$ = softmax with temperature τ
- α, β = weighting coefficients (0.5 each)
- $\tau = 3.0$

Secondly, feature-based distillation was implemented to extract features from intermediate teacher layers. The TeacherFeatureExtractor class registers hooks on the low, mid, and high layers of ResNet50. The student simultaneously outputs its corresponding feature maps from its encoder. Cosine-similarity loss is then calculated on each level and combined with weights w_i as shown:

$$L_{feature} = \sum_{i \in \{low, mid, high\}} w_i (1 - \cos(f_s^{(i)}, f_t^{(i)}))$$

Where:

- $f_s^{(i)}, f_t^{(i)}$ = student and teacher feature maps at level i
- $w_i = \{0.2, 0.3, 0.5\}$

This loss is then added to the ground truth loss L_{CE}

$$L_{total} = L_{CE} + \lambda \cdot L_{feature}$$

$\lambda=0.1$.

After each epoch, validation mIoU is computed, and the model with the best value is saved as `best_method`.

Hyperparameter

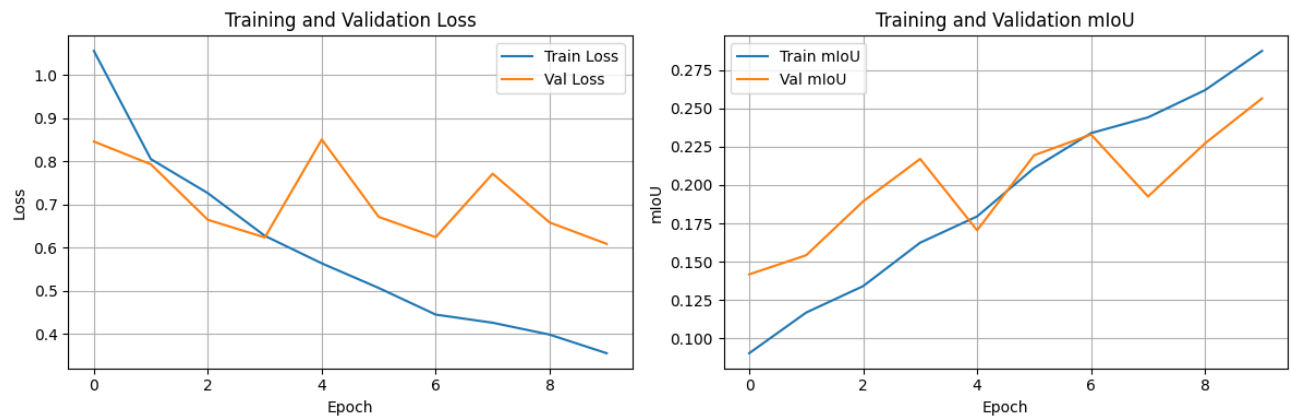
Parameter	Value	Purpose
Dataset	Pascal VOC 2012 (train / val)	21 semantic classes
Input Size	256×256	Standardized batch size
Batch Size	8	Memory and speed trade-off
Optimizer	Adam	Stable updates
Epochs	10	Per distillation method
α, β, τ	0.5, 0.5, 3.0	Response KD weights
Feature Weights	0.2, 0.3, 0.5	Low/mid/high levels

Best validation mIoU (from CPU training runs):

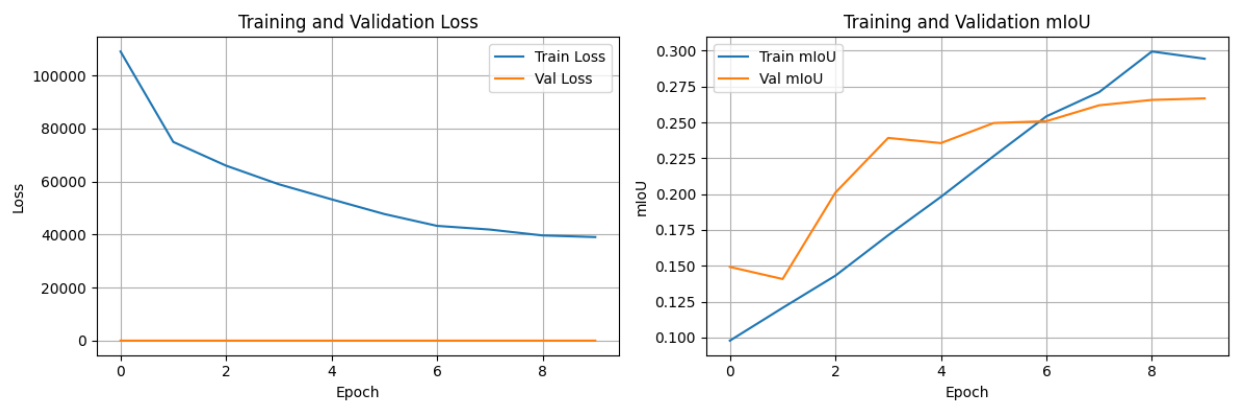
- Baseline: 0.2565
- Response-KD: 0.2667
- Feature-KD: 0.2525

Below are the loss plots for the three student models. The response KD loss is very large (~100000) because the summed Kullback-Leibler divergence over all pixels produces very large values. This did not affect training stability because the learning rate remained constant.

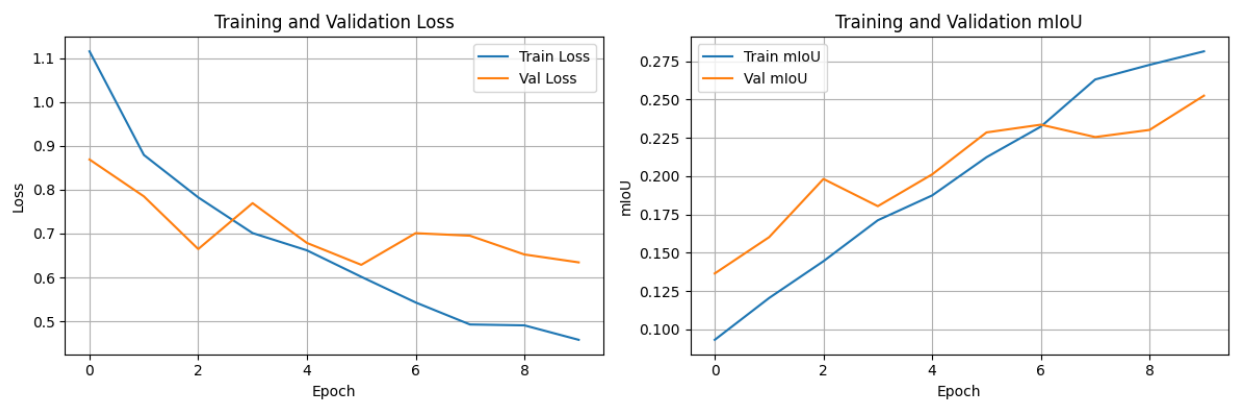
No KD loss plot:



Response KD loss plot:



Feature KD loss plot:



Experiments

Quantitative results from testing on a CUDA T4 GPU (Google Colab):

Knowledge Distillation	mIoU	# Parameters	Inference (ms/img)
None	0.4197	6.9 M	5
Response	0.4398	6.9 M	4.75
Feature	0.4195	6.9 M	4.56

Training duration (10 epochs each, batch = 8, 256×256 images):

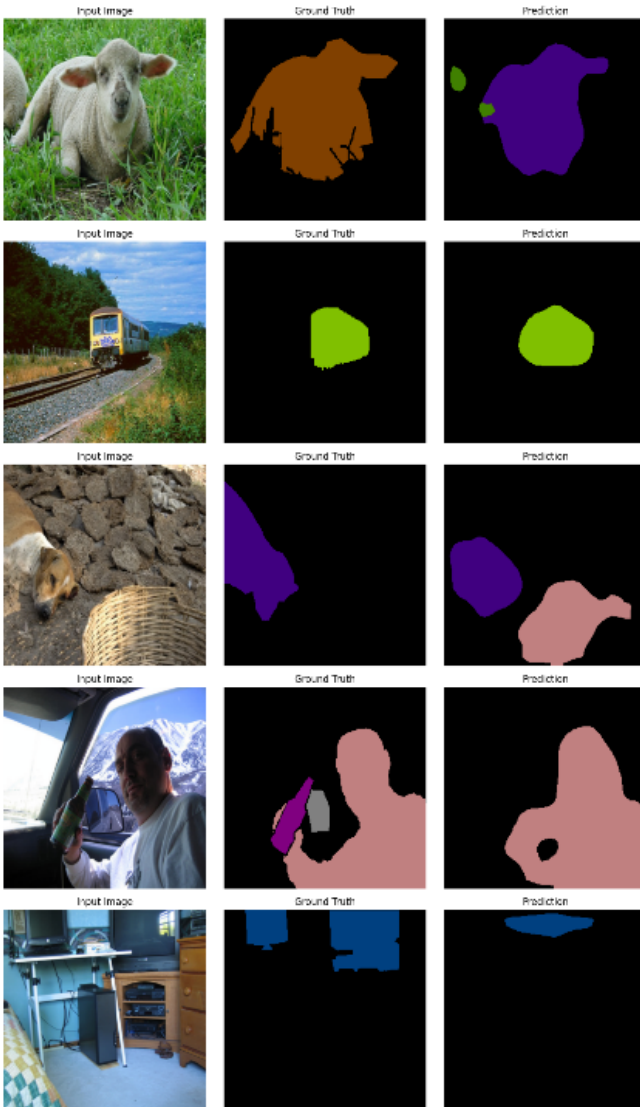
Method	Approx. time per epoch (CPU)	Total train time (10 epochs)
Without distillation	≈ 9 – 11 min	≈ 1 h 35 min
Response-based KD	≈ 50 – 52 min	≈ 8 h 25 min
Feature-based KD	≈ 49 – 51 min	≈ 8 h 20 min

Testing (GPU, batch = 8, 256×256):

Model	Inference time (ms / image) (from test.py)	Dataset size (test = 1449 images)
FCN-ResNet50 (teacher)	21.76 ms	≈ 31 s total
Student (baseline)	5.00 ms	≈ 7 s total
Student (response-KD)	4.75 ms	≈ 6.9 s total
Student (feature-KD)	4.56 ms	≈ 6.6 s total

Validation mIoU while training using a CPU was lower at 0.26. The final GPU test uses the same dataset split and transforms but benefits from much faster hardware and parallel processing. importantly, it evaluates the saved best checkpoints with the standardized testing loop and can yield higher mIoU than the CPU validation done in training.

test.py saves visualisations for the model with the best prediction, which in this case is response knowledge distillation. Below is an example of this:



Performance

The performance for the response-based KD was expected, as it improved accuracy over the baseline. However, feature-based KD decreased the mIoU of the student model, which went against our expectation. Overall, the mIoU during training on the CPU was much lower than the final GPU test, which is explainable by runtime differences and the fact that the GPU testing evaluated the best saved checkpoints after 10 epochs.

- **Compression & speed:** The student is $\sim 5\times$ faster than the teacher on GPU ($\approx 4.6\text{--}5.0$ ms/img vs 21.76 ms/img) and $\sim 5\times$ smaller in parameters (6.94 M vs 35.32 M), making it suitable for resource-constrained deployments.
- **Accuracy:** Response-based KD delivered the best student accuracy in this run (mIoU 0.4398 vs baseline 0.4197). Feature-based KD did not beat baseline (0.4195) under the current 10-epoch budget and weights.

As to why feature-based KD underperformed, it can be boiled down to three possible reasons:

- Budget: feature-KD typically needs more epochs to pay off
- Alignment: student MobileNet taps (16/24/576 ch) vs teacher ResNet taps may not be perfectly compatible
- Weighting: λ , w_{low} , w_{mid} , and w_{high} may need tuning

Hallucinations

To ensure code generated by the LLM (Claude) was not hallucinated, the code would immediately be checked line-by-line and then run in Colab to ensure there were no errors. The code generated would also be compared to class concepts to ensure that it is following the same structure and is correctly following the requirements. The Claude conversation link is below:

<https://claude.ai/share/e1528e8b-2aea-43f4-ad39-24a6b88f08d7>